Chapter 1
Number Systems

Agenda

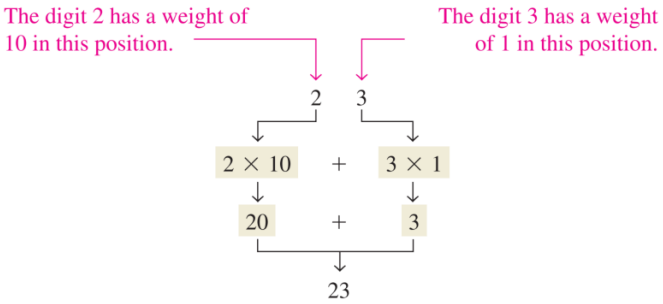
- Some preliminaries on how to convert between decimal, binary and hex.
- 8-bit unsigned number
- 8-bit signed number
- 2's complement: how to do 2'complement quickly?
- Overflow in signed arithmetic
- Multiplication
- Division

Positional Number Systems

- Positional system → "value" is based on the symbol and its position in the number
- Base-r number system
  - r unique symbols
  - represent 0 to r-1
  - e.g., decimal has 10 symbols, 0 to 9
    e.g., binary has 2 symbols, 0 to 1
    e.g., hexadecimal has 16 symbols, 0 to 9 and A to F

Base-r number systems

- The position of each digit indicates the magnitude of the quantity presented and can be assigned a weight, e.g., in decimal,



- In general, a number XYZ in base-r number system has the value of:

$$X \times r^2 + Y \times r^1 + Z \times r^0$$

- Positional systems must have the concept of "zero", e.g., in decimal, 26 or 206?

Notation systems

	written	ARM	Intel
decimal	10_{10}	10	10
hexadecimal	A_{16}	0xA	0AH
binary	1010_2	0b1010	1010B

- Computer system bases:
  2 (binary) and 16 (hexadecimal)
- Assembly language uses either
  a prefix notation system (e.g., ARM) or
  a suffix notation system (e.g., Intel)
- Syntax varies by assembly language *and*
  the development environment

Conversions between number systems

1) any base to decimal
   - expand the number into positional notation and evaluate

e.g., $110_2 = 1x2^2 + 1x2^1 + 0x2^0$

$= 4+2+0 = 6_{10}$

e.g., $A01_{16} = Ax16^2 + 0x16^1 + 1x16^0$

$= 10x256 + 0 + 1 = 2561_{10}$

🧠 Micro-Challenge: Positional Value

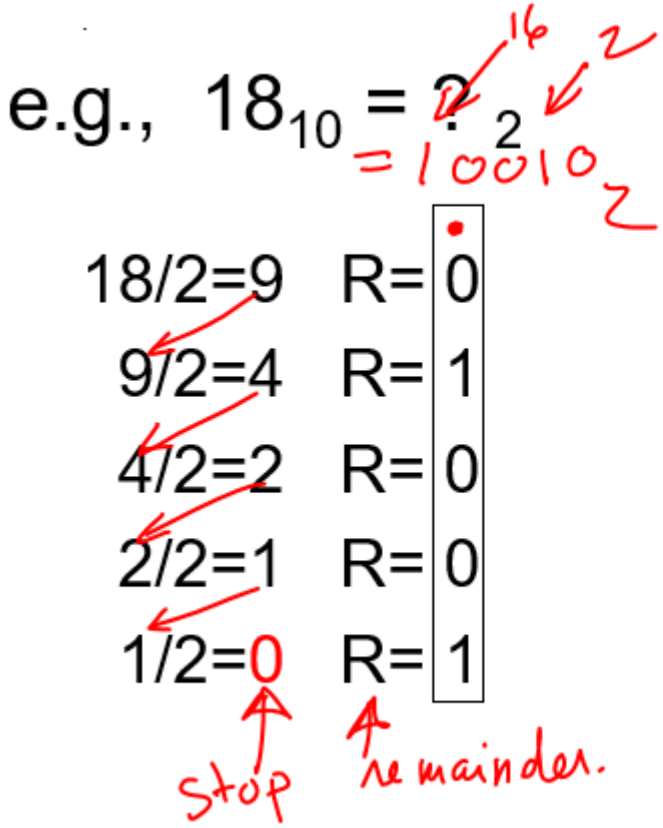
Without writing anything down:

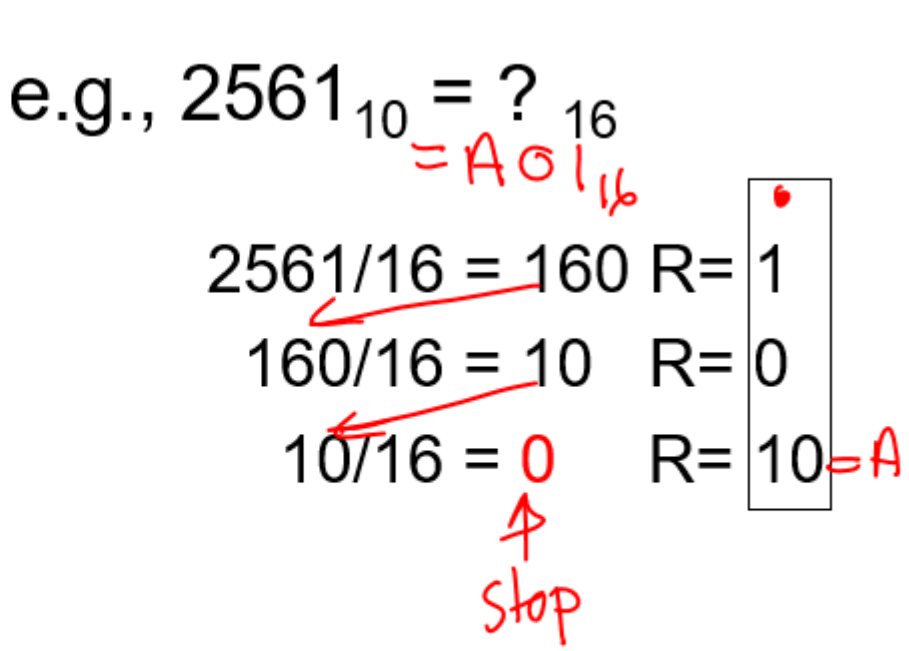
What is the decimal value of 10101_2 ?

Conversions between number systems

2) b) decimal to any base r

→ *integer portion – successive divide by r*





🧠 Micro-Challenge: Convert Together

- Turn to your neighbour:

- Convert 19_{10} to binary (60 seconds)

Conversions between number systems

3) binary ↔ hexadecimal

4 binary digits ≡ 1 hexadecimal digit (a nibble)

e.g. $001110110011_2 = ?_{16}$

3 B 3

e.g. $46A1_{16} = ?_2$

0100011010100001

🧠 Micro-Challenge: Nibble Mapping

- Convert without calculator:

- 0x3F → binary

(Think in nibbles)

Number Representation

- A number is stored in a register or a memory location
- Assembly language fixes the length of the register and the memory location to a specific length *n*
- For ARM,
  n = 32 bits

Number Representation

1) Unsigned
   - number is assumed to be non-negative and no bits are used to represent the sign

     e.g., $00000001_2 = 1_{10}$

     $10000001_2 = 1 \times 2^7 + 1 \times 2^0$

     $= 128 + 1 = 129_{10}$
   - Range of an n-bit register:

     00… 00 → 11…11

Most significant bit Least significant bit

$0 \rightarrow 2^n - 1$

Number Representation

2) Signed
   - numbers must have the following characteristics to be useful for calculations:
   1. positive and negative numbers
   2. sign test - easy way to determine if the number is positive or negative
   3. zero test - easy way to identify zero with preferably only one representation of zero.
   4. easy implementation of arithmetic operations.

Number Representation

Ways to represent signed numbers:

1. **sign-magnitude**

in general: SXX … X e.g., $00000001_2 = +1$
$10000001_2 = -1$

- both sign and magnitude are immediately obvious
- two zeros (i.e, 00000000 and 10000000)
- Addition and subtraction require different behaviors depending on the sign bit (e.g., adding two positive numbers require different operations from adding numbers with opposite signs)

Number Representation

2. Two's complement

- Positive number: Same as unsigned number representation
- Negative number:
  - Step 1: Write the magnitude of the number as an unsigned number
  - Step 2: Invert each bit
  - Step 3: Add 1 to it

Example – Decimal → Signed 8-bit

What is the 8-bit signed representation of -127_{10}?

Step 1:

- Magnitude = $127_{10} = 01111111_2$

Step 2:

- Invert → 10000000

Step 3:

- +1 → **10000001**

Example – Decimal → Signed 8-bit

- Quick way to do 2's complement in hex

  Step 1:

  – Magnitude = 01111111 = 7F

  Step 2:

  – Inversion is equivalent to subtracting F from the most (MSN) and least significant nibble (LSN):

  – MSN: F-7 = 8; LSN: F-F = 0. The result is 80.

  Step 3:

  – 80 + 1 = 81

Example – Signed 8-bit → Decimal

- What is the value of the 8-bit signed number 0xFF expressed in decimal?
  1. Recognize that this is a negative number
     - MSB is 1. (F → 1111)
  2. Two complements to find magnitude
     - Invert FF → 00
     - +1 → 01
     - Magnitude = 01
     - Answer: 0xFF → -1

🧠 Micro-Challenge: Signed 8-bit

- What is the value of the 8-bit signed number 0x80 expressed in decimal?

8-bit signed number representation

Decimal	Binary	Hex
+127	0111 1111	7F
.....	.....	.....
+2	0000 0010	02
+1	0000 0001	01
0	0000 0000	00
-1	1111 1111	FF
-2	1111 1110	FE
....	....	....
-127	1000 0001	81
-128	1000 0000	80

32-bit signed number representation

Decimal	Binary	Hex
-2,147,483,648	10000000000000000000000000000000	80000000
-2,147,483,647	10000000000000000000000000000001	80000001
-2,147,483,646	10000000000000000000000000000010	80000002
...	...	...
-2	11111111111111111111111111111110	FFFFFFFE
-1	11111111111111111111111111111111	FFFFFFFF
0	00000000000000000000000000000000	00000000
+1	00000000000000000000000000000001	00000001
+2	00000000000000000000000000000010	00000002
...	...	...
+2,147,483,646	01111111111111111111111111111110	7FFFFFFE
+2,147,483,647	01111111111111111111111111111111	7FFFFFFF

- Range of signed number:

$$-(2^{n-1}) \text{ to } (2^{n-1} - 1)$$

where n is the number of bits

Two’s complement

- one zero (i.e., 00000000)
- Sign immediately obvious: Most significant bit = 1 → negative; Most significant bit = 0 → positive
- Addition and subtraction require the same operation

Addition of signed numbers

Both numbers positive:

00000111	7
+ 00000100	+ 4
00001011	11

The sum is positive and is therefore in true (uncomplemented) binary.

Positive number with magnitude larger than negative number:

	00001111	15
	+ 11111010	+ −6
Discard carry → 1	00001001	9

The final carry bit is discarded.

Negative number with magnitude larger than positive number:

00010000	16
+ 11101000	+ −24
11111000	−8

The sum is negative and therefore in 2's complement form.

Both numbers negative:

	11111011	−5
	+ 11110111	+ −9
Discard carry → 1	11110010	−14

The final carry bit is discarded. The sum is negative and therefore in 2's complement form.

Overflow condition

- Overflow occurs when two numbers are added and the number of bits required to represent the sum exceeds the number of bits available in the representation
- Can happen when both numbers are positive or both numbers are negative
- Overflow is indicated if:
  - Correct result is out of the range between -2^{n-1} and $2^{n-1}-1$, where n is the number of bits (e.g., -128_{10} and 127_{10} for 8 bit)
  - the sum of two positive numbers is a negative number
  - the sum of two negative numbers is a positive number

125_{10}	01111101	Correct result = 183_{10}
$+\ 58_{10}$	00111010	→ Out of range
-73_{10}	10110111	

Five scenarios

1. +ve + -ve → No overflow
2. +ve + +ve:
   - 2a: if answer less than or equal to max. (for 8-bit max. is 127 in decimal) – results must be +ve → No overflow
   - 2b: if answer larger than max – results must be –ve → Overflow
3. -ve + -ve:
   - 3a: if answer greater than or equal to min. (for 8-bit max. is -128 in decimal) – results must be –ve → No overflow
   - 3b: if answer smaller than min – results must be +ve → Overflow

🧠 Micro-Challenge: Overflow

Determine whether or not overflow occurs in each example.

Example 1

3C
+32
6E

Example 2

32
+EC
11E

Example 3

CE
+D8
A6

Example 4

9C
+CE
16A

Example 5

64
+3C
A0

Subtraction

- Subtraction is a special case of addition
- $a - b = a + (-b)$
- $-b$ is obtained by taking the 2's complement of b

```
 3F      3F
- 23    +DD   (2's complement) (MSN: F-2 = D; LSN: F-3 = C; DC+1 = DD)
        ¹1C    Carry = 1; Discard carry from result;
Result = 1C

 23      23
- 3F    +C1   (2's complement) (MSN: F-3 = C; LSN: F-F = 0; C0+1 = C1)
         E4    Carry = 0
Result = E4
```

- Note that Carry = 1 if result is positive and Carry = 0 if result is negative
- This is important for establishing whether a and b is greater

Sign extension of signed number

- Positive number: e.g., extending the 8-bit number of 56_{16} to a 32-bit number will result in 00000056_{16}
- Negative number: e.g., extending the 8-bit number of 82_{16} to a 32-bit number will result in $FFFFFF82_{16}$
  - 82_{16} is $-7E_{16}$
  - With 32-bit, this number is represented as the 2'complement of $0000007E_{16}$ → $FFFFFF82_{16}$
- Thus, signed extension involves copying the signed bit to the upper 24 bits of the 32-bit register.

Signed extension of signed number

Similar in extending to other number of bits > 8: Copy the signed bit into the additional bits in the new formats

- 82_{16}, when extending to 16 bits, becomes $FF82_{16}$
- 56_{16}, when extending to 16 bits, becomes 0056_{16}

Micro-Challenge: Sign Extension

For each 8-bit signed number below, perform sign extension and write the 16- and 32-bit results in hexadecimal.

8-bit	16-bit	32-bit
25 (MSB = 0)		
7A (MSB = 0)		
92 (MSB = 1)		
80 (MSB = 1)		

Multiplication

How do you multiply decimal numbers?

32 → Multiplicand
x12 → Multiplier
64 → Multiply first digit 2 → write down product
32 → Multiply second digit 1 → shift left, write down product
384 → Add together partial product

Multiplication

- Multiplication of unsigned number by partial product

10×13	Multiplier = 1101_2	
	Multiplicand = 1010_2	
1010		
1101		
1010	Step 1 first multiplier bit	= **1,** write down multiplicand
0000	Step 2 second multiplier bit	= **0,** write down zeros shifted left
1010	Step 3 third multiplier bit	= **1,** write down multiplicand shifted left
1010	Step 4 fourth multiplier bit	= **1,** write down multiplicand shifted left
10000010	Step 5 add together four partial products	

- n-bit × n-bit → 2n-bit result

High-speed multiplication

FIGURE 2.3 An algorithm for multiplication

Step a. Set a counter to *n*.

Step b. Clear the 2*n*-bit partial product register.

Step c. Examine the rightmost bit of the multiplier (initially the least-significant bit). This bit is underlined in Table 2.3. If it is one, add the multiplicand to the *n* most-significant bits of the partial product. Record the carry (C) flag

Step d. Shift the partial product one place to the right. Shift in the carry (C) flag.

Step e. Shift the multiplier one place to the right (the rightmost bit is, of course, lost).

Step f. Decrement the counter and repeat from step c until the count is 0 after *n* cycles. The product is in the partial product register.

© Cengage Learning 2014

TABLE 2.3 Mechanizing Unsigned Multiplication from Figure 2.3

		Multiplier = 1101_2	Multiplicand = 1010_2		
Cycle	Step	Counter	Multiplier	C	Partial Product
	a and b	4	1101		00000000
1	c	4	1101	0	10100000
1	d and e	4	0110	shift in C	01010000
1	f	3	0110		01010000
2	c	3	0110	0	01010000
2	d and e	3	0011	shift in C	00101000
2	f	2	0011		00101000
3	c	2	0011	0	11001000
3	d and e	2	0001	shift in C	01100100
3	f	1	0001		01100100
4	c	1	0001	1	00000100
4	d and e	1	0000	shift in C	10000010

+10100000

+00000000

+10100000

+10100000

10×13

$$\begin{array}{r} 1010 \\ \times\ 1101 \\ \hline 1010 \\ 0000 \\ 1010 \\ 1010 \\ \hline 10000010 \end{array}$$

Note: Result = 10000010

High-speed multiplication

TABLE 2.3 Mechanizing Unsigned Multiplication from Figure 2.3

		Multiplier = 1101_2	Multiplicand = 1010_2		
Cycle	Step	Counter	Multiplier	C	Partial Product
	a and b	4	1101		00000000
1	c	4	1101	0	10100000
1	d and e	4	0110	shift in C	01010000
1	f	3	0110		01010000
2	c	3	0110	0	01010000
2	d and e	3	0011	shift in C	00101000
2	f	2	0011		00101000
3	c	2	0011	0	11001000
3	d and e	2	0001	shift in C	01100100
3	f	1	0001		01100100
4	c	1	0001	1	00000100
4	d and e	1	0000	shift in C	10000010

+10100000

+00000000

+10100000

+10100000

10×13

$$\begin{array}{r} 1010 \\ \times\ 1101 \\ \hline 1010 \\ 0000 \\ 1010 \\ 1010 \\ \hline 10000010 \end{array}$$

Note: Result = 10000010

Iteration	Partial Product at the end of the iteration
1	01010000
2	00101000
3	01100100
4	10000010

🧠 Micro-Challenge: High-Speed Multiplication

- Determine the partial products of the multiplication between the following binary multiplier and multiplicand after each of the four iterations of the high-speed multiplication algorithm. Express your answers in 8-bit binary format.

- Multiplicand: 1101
- Multiplier: 1010

Iteration	Partial Product at the end of the iteration
1	
2	
3	
4	

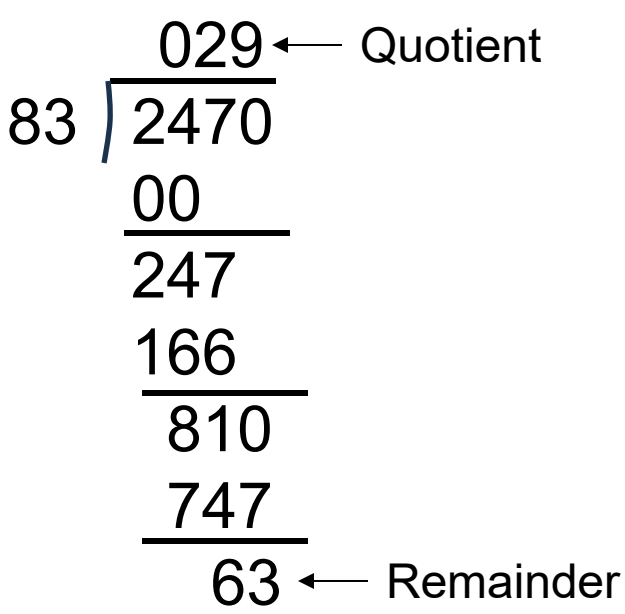
🧠 Micro-Challenge: High-Speed Multiplication

- Multiplicand: 1101
- Multiplier: 1010

Iteration	Partial Product at the end of the iteration	
Initial	00000000	+00000000, then shift C=0
1	00000000	+11010000, then shift C=0
2	01101000	+00000000, then shift C=0
3	00110100	+11010000, then shift C=1
4	10000010	

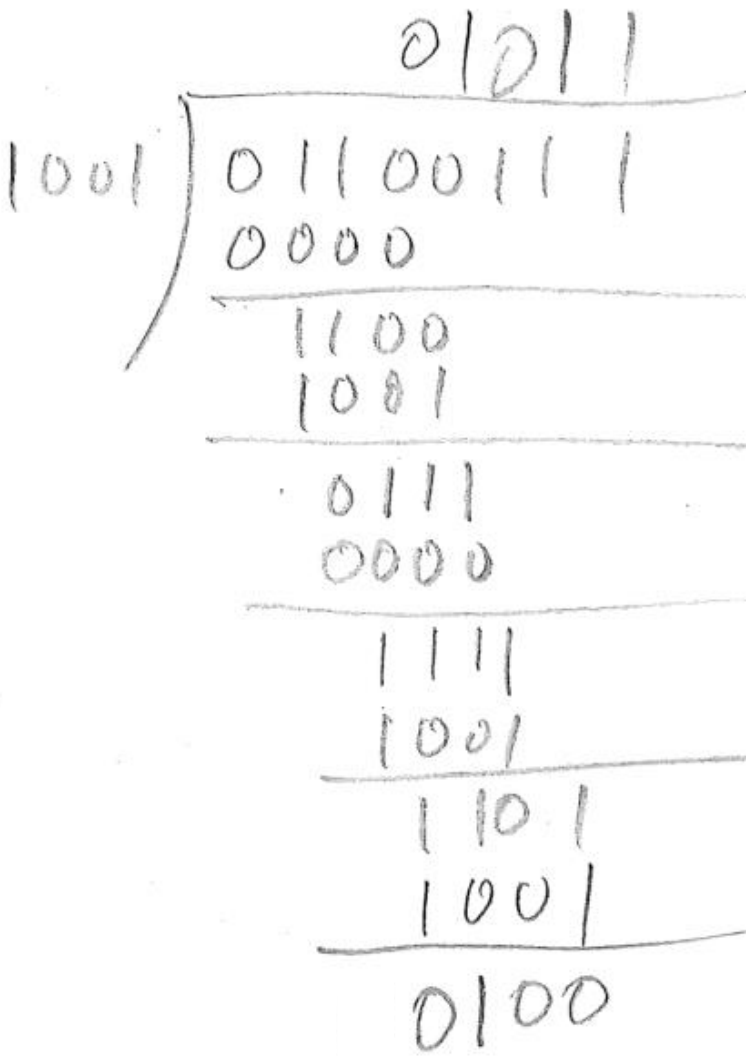
Division

- How do you divide decimal numbers?



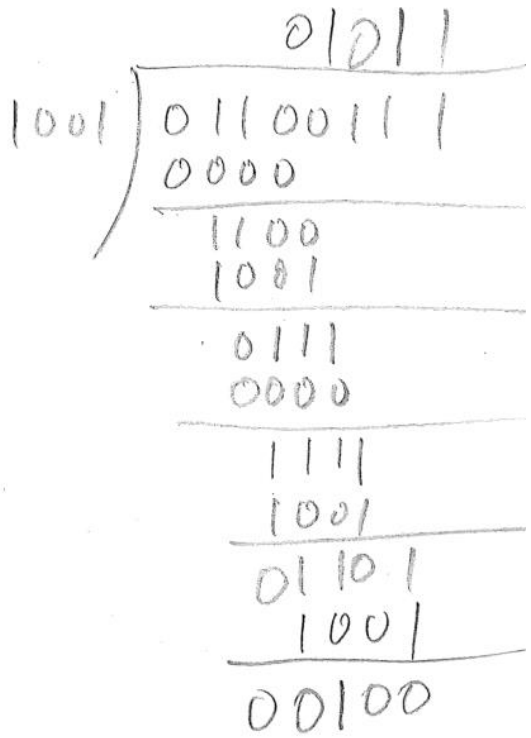
- Dividend = 2470
- Divisor = 83
- Quotient = 29
- Remainder = 63

Division



- Dividend = 103_{10} = 01100111_2
- Divisor = 9_{10} = 1001_2
- Expected Result:
  - Quotient = 1011_2 = 11_{10}
  - Remainder = 100_2 = 4_{10}

Restoring division

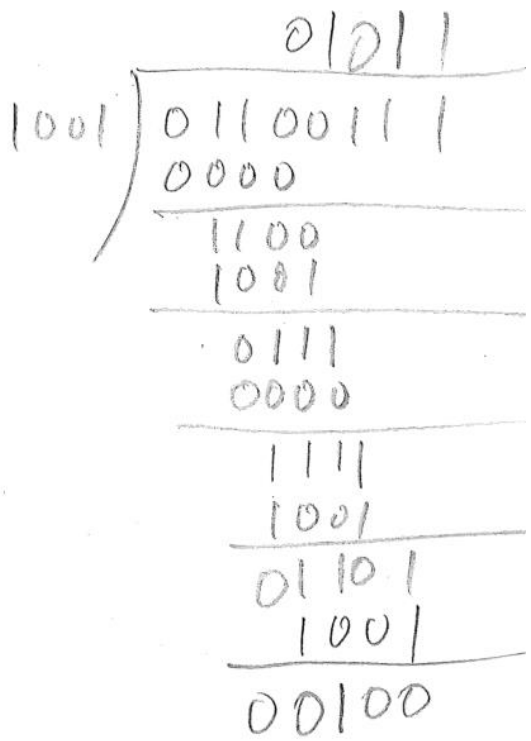


Description	Partial Dividend	Divisor	Quotient
	01100111	00001001	00000000
Align	01100111	10010000	00000000
Subtract divisor from partial dividend	-00101001	10010000	00000000
Restore divisor, shift 0 in quotient	01100111	10010000	00000000
Test for end			
Shift divisor one place right	01100111	→ 01001000	00000000
Subtract divisor from partial dividend	00011111	01001000	00000000
Result positive—shift in 1 quotient	00011111	01001000	00000001
Test for end			
Shift divisor one place right	00011111	→ 00100100	00000001
Subtract divisor from partial dividend	−00000101	00100100	00000001
Restore divisor, shift in 0 in quotient	00011111	00100100	00000010
Test for end			
Shift divisor one place right	00011111	→ 00010010	00000010
Subtract divisor from partial dividend	00001101	00010010	00000010
Result positive—shift in 1 in quotient	00001101	00010010	00000101
Test for end			
Shift divisor one place right	00001101	→ 00001001	00000101
Subtract divisor from partial dividend	00000100	00001001	00000101
Result positive—shift in 1 in quotient	00000100	00001001	00001011
Test for end			

Remainder

End: Divisor aligns with LSB of dividend

Restoring division



Description	Partial Dividend	Divisor	Quotient
	01100111	00001001	00000000
Align	01100111	10010000	00000000
Subtract divisor from partial dividend	-00101001	10010000	00000000
Restore divisor, shift 0 in quotient	01100111	10010000	00000000
Test for end			
Shift divisor one place right	01100111	→ 01001000	00000000
Subtract divisor from partial dividend	00011111	01001000	00000000
Result positive—shift in 1 quotient	00011111	01001000	00000001
Test for end			
Shift divisor one place right	00011111	→ 00100100	00000001
Subtract divisor from partial dividend	−00000101	00100100	00000001
Restore divisor, shift in 0 in quotient	00011111	00100100	00000010
Test for end			
Shift divisor one place right	00011111	→ 00010010	00000010
Subtract divisor from partial dividend	00001101	00010010	00000010
Result positive—shift in 1 in quotient	00001101	00010010	00000101
Test for end			
Shift divisor one place right	00001101	→ 00001001	00000101
Subtract divisor from partial dividend	00000100	00001001	00000101
Result positive—shift in 1 in quotient	00000100	00001001	00001011
Test for end			

Remainder

End: Divisor aligns with LSB of dividend

Iteration	Partial Dividend	Quotient
1st	01100111	00000000
2nd	00011111	00000001
3rd	00011111	00000010
4th	00001101	00000101
5th	00000100	00001011

Micro-Challenge: Restoring Division

- The restoring division is used for a division operation with:

- Dividend: 01011010_2
- Divisor: 1011_2

(Expected: 90/11 = 8R2)

- Determine the partial dividend and the quotient after each iteration of the restoring division operation. Assume the registers storing the partial dividend and the quotient are 8-bit

Iteration	Partial Dividend	Quotient
1st		
2nd		
3rd		
4th		
5th		

Micro-Challenge: Restoring Division

- Dividend: $01011010_{2,}$ Divisor: 1011_2

Iteration	Partial Dividend	Quotient
	01011010 -10110000 → -ve	
1st	01011010	00000000
2nd		
3rd		
4th		
5th		

Micro-Challenge: Restoring Division

- Dividend: $01011010_{2,}$ Divisor: 1011_2

Iteration	Partial Dividend	Quotient
	01011010 -10110000 → -ve	
1st	01011010	00000000
	01011010 -01011000 00000010	
2nd	00000010	00000001
	00000010 -00101100 → -ve	
3rd	00000010	00000010
	00000010 -00010110 → -ve	
4th	00000010	00000100
	00000010 -00001011 → -ve	
5th	00000010	00001000